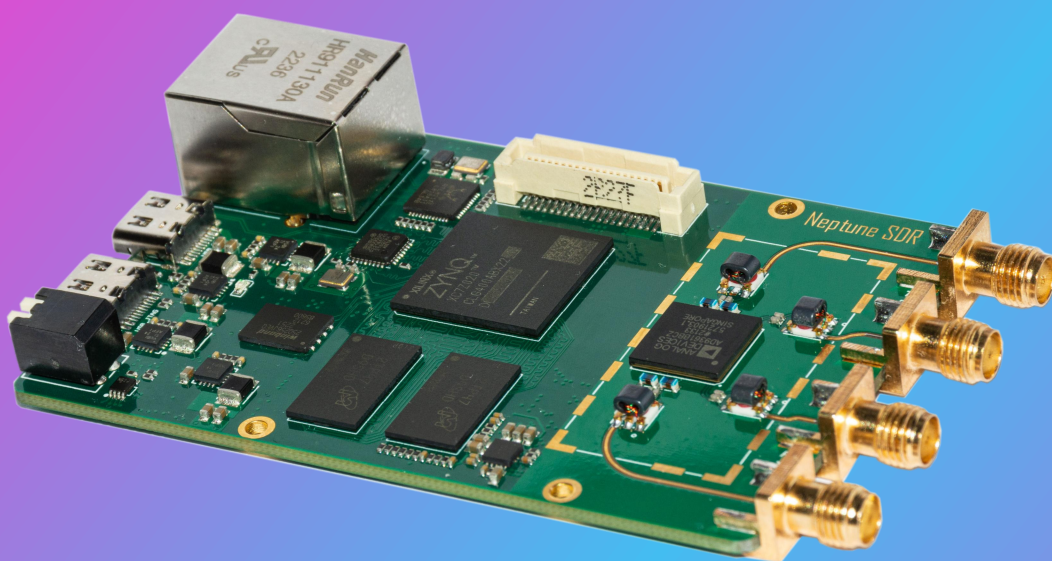


NeptuneSDR

Zynq7020 AD9361 开发平台

K210 用户手册



目录

版权声明	3
免责声明	4
中国微功率短距离设备声明	5
一、开发板简介	7
二、ZYNQ 芯片	9
三、DDR 芯片	10
四、QSPI Flash	10
五、PS 系统时钟源	11
六、结构图	12
七、BTB 连接器管脚定义	12
八、USB 2.0 接口	13
九、千兆以太网接口	13
十、UART 接口	14
十一、SD 卡槽	14
十二、AD9361	15

版权声明

涅普科技拥有本文档的版权所有，未经涅普科技正式书面授权许可，任何公司、团体、网站、媒体或个人均不得以任何方式或形式修改、分发、复制或摘编本文档的任何内容。这些内容包括但不限于文字、图片、图表、表格等。本文档的知识产权归涅普科技所有，任何人或组织不得通过任何渠道将其用于商业用途。涅普科技保留追究侵权者法律责任的权利。

我们非常重视文档的知识产权，这是我们对知识产权法律的尊重和对创作者的尊重。我们希望所有使用本文档的人都能尊重知识产权，遵守上述声明，避免因侵权行为而导致的不必要的法律纠纷和风险。同时，我们也鼓励大家尊重自己的知识产权，创造更多有价值的作品。

希望能够通过我们的努力，提高大家对知识产权的认识和尊重，共同维护知识产权的合法权益，让知识产权得到更好的保护和发展。感谢您的合作和支持！

免责声明

本产品仅供研究学习使用，请购买方在合法合规范围内使用本产品。购买方使用本产品所引起的任何纠纷和造成的一切后果，均由购买方自行承担，与涅普科技无关。特此声明！

本产品中所提供的代码、软件、文档等资料，涅普科技不提供任何类型的担保，无论明示或暗示，包括但不限于适用于特定用途的保证，使用者需承担全部风险。我们深知技术的不确定性，也深知用户在使用中可能遇到的困难，因此我们一直在为用户提供更多的技术支持，帮助用户克服技术难题，也凭借着专业技术，为用户提供更好的使用体验。

未经涅普科技授权或相关主管部门认证，擅自更改或修改本产品，将导致无线合规无效，使您失去操作本产品的权限。我们建议用户在使用本产品前，详细了解相关法规法律，以免发生不必要的麻烦。

我们希望用户能够合法合规地使用本产品，共同构建一个良好的技术生态环境。如果您在使用过程中遇到任何问题或有任何建议，欢迎随时联系我们，我们将竭诚为您服务。

中国微功率短距离设备声明

本产品具备《中华人民共和国无线电管理条例》规定的微功率短距离无线电发射能力，根据“工业和信息化部公告 2019 年第 52 号”的要求，现注明如下：

（一）产品符合“微功率短距离无线电发射设备目录和技术要求”中通用微功率设备 F 类设备技术要求，用于研究学习用途。产品未配置天线，控制开关等信息请参考使用手册。

（二）不得擅自改变使用场景或使用条件、扩大发射频率范围、加大发射功率（包括额外加装射频功率放大器），不得擅自更改发射天线；

（三）不得对其他合法的无线电台（站）产生有害干扰，也不得提出免受有害干扰保护；

（四）应当承受辐射射频能量的工业、科学及医疗（ISM）应用设备的干扰或其他合法的无线电台（站）干扰；

（五）如对其他合法的无线电台（站）产生有害干扰时，应立即停止使用，并采取措施消除干扰后方可继续使用；

（六）在航空器内和依据法律法规、国家有关规定、标准划设的射电天文台、气象雷达站、卫星地球站（含测控、测距、接收、导航站）等军民用无线电台（站）、机场等的电磁环境保护区域内使用微功率设备，应当遵守电磁环境保护及相关行业主管部门的规定；

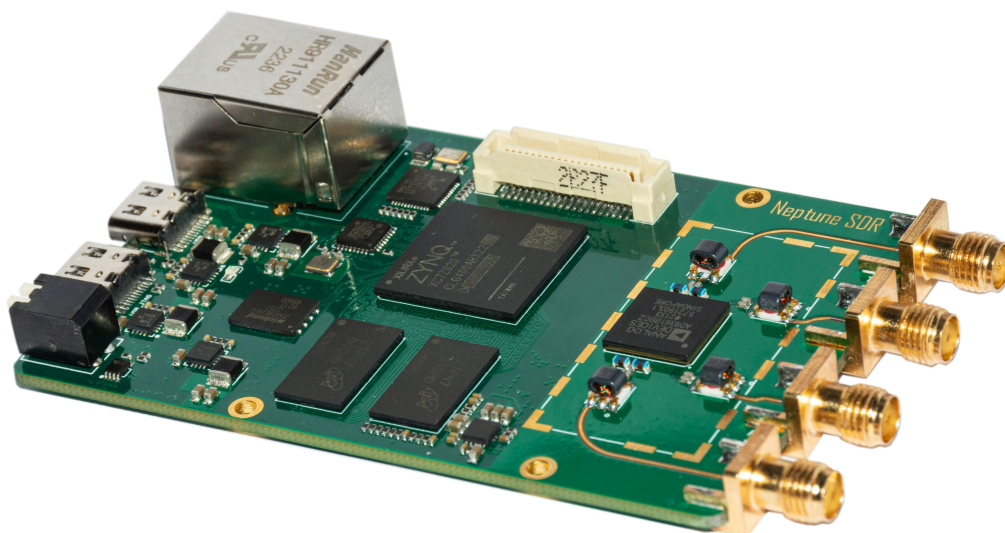
（七）禁止在以机场跑道中心点为圆心、半径 5000 米的区域内使用各类模型遥控器；

（八）微功率设备的工作电压为 5 伏，在常温下正常工作。

涅普科技有限责任公司基于 XILINX Zynq7020+ AD9361 开发平台的开发板（型号：K210）2022 款正式发布了，为了让您对此开发平台可以快速了解，我们编写了此用户手册。

这款 SDR 开发平台采用一体式设计模式，方便用户对开发板二次开发利用。开发板使用 XILINX Zynq 7020+ AD9361 芯片的解决方案，它采用 Processing System(PS) + Programmable Logic(PL) 技术将四核 ARM Cortex-A9 和 FPGA 可编程逻辑集成在一颗芯片上。另外核心板上 PS 端带有 2 片共 1GB 高速 DDR3 SDRAM 芯片和 1 片 256Mb 的 QSPI FLASH 芯片；开发板上 PL 端负责与 AD9361 芯片进行通信，模式 LVDS 与 CMOS 可选。

在开发板设计上我们为用户扩展了丰富的外围接口，比如 1 个 BTB 接口、1 个 USB2.0 OTG 接口、1 路千兆以太网接口、1 路 UART 接口、1 路 SD 卡接口、1 路 JTAG 接口等等。满足用户各种高速数据交换，数据存储，视频传输处理，深度学习，人工智能以及工业控制的要求，是一款“专业级”的 ZYNQ SDR 开发平台。为高速数据传输和交换，数据处理的前期验证和后期应用提供了可能。相信这样的一款产品非常适合从事通信算法开发的学生、工程师等群体。



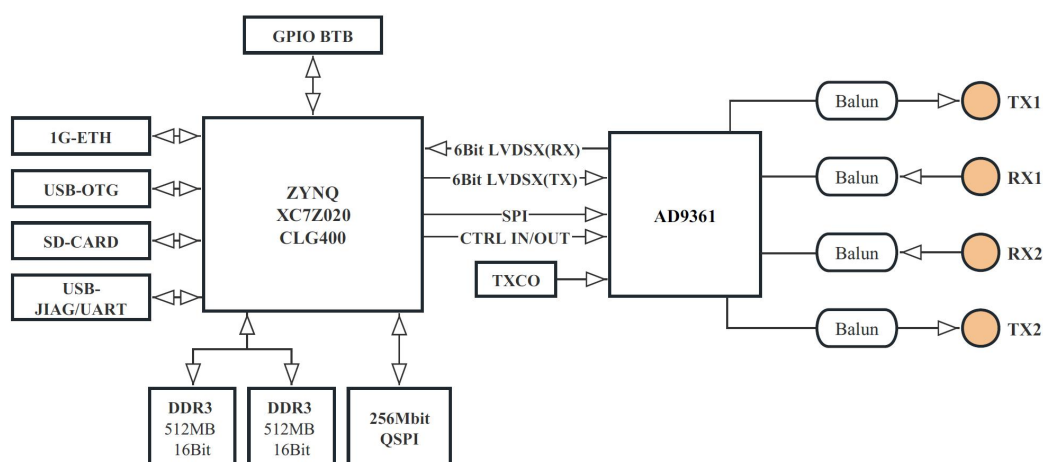
一、开发板简介

在这里，对这款 K210 SDR 开发平台进行简单的功能介绍。

开发板主要由 XC7Z020 + 2 个 DDR3 + AD9361 + 1 个 QSPI FLASH 的最小系统构成。主芯片采用 Xilinx 公司的 Zynq 7000 系列的芯片，型号为 XC7Z020-1CLG400I。Zynq7020 芯片可分成处理器系统部分 Processor System (PS) 和可编程逻辑部分 Programmable Logic (PL)。在 Zynq7020 芯片的 PS 端挂了 2 片 DDR3，每片 DDR3 容量高达 512M 字节，使得 ARM 系统和 FPGA 系统能独立处理和存储的数据的功能。PS 端的 1 片 256Mb 的 QSPI FLASH 用来静态存储 Linux 的操作系统、文件系统及用户数据。

开发板扩展了丰富的外围接口，其中包含 1 个 BTB 接口、1 个 USB2.0 OTG 接口、1 路千兆以太网接口、1 路 UART 接口、1 路 SD 卡接口、1 路 JTAG 接口和一些按键 LED。

下图为整个开发系统的结构示意图：



通过这个示意图，我们可以看到，我们这个开发平台所能含有的接口和功能。

Zynq7020 最小系统

由 Zynq7020+1GB DDR3 (PS) + 256Mb QSPI FLASH 组成，另外有 1 个晶振提供时钟，一个单端 33.3333MHz 晶振提供给 PS 系统。

USB2.0 接口

USB 接口类型为 TYPE C，工作模式为 OTG 模式。用于连接外部的 USB 主机，比如连接

PC、树莓派等上位机设备。

千兆以太网接口

1 路 10/100M/1000M 以太网 RJ45 接口，连接于 PS 端。用于和电脑或其它网络设备进行以太网数据交换。

USB DEBUG 接口

USB 接口类型为 TYPE C，该 USB DEBUG 接口提供 1 路 Uart 和 1 路 JTAG，用于和电脑通信，方便用户调试。接口芯片采用 FT2232 芯片。

Micro SD 卡座

1 路 Micro SD 卡座，用于存储操作系统镜像和文件系统。

BTB 扩展口

1 个用户自定义 BTB 的扩展口，可以外接用户自定义子板或者我们提供的各种 BTB 模块（HDMI 输入输出模块，摄像头模块，以太网模块等等）。

JTAG 调试口

1 路 JTAG 调试口用于工厂调试，与 FT2232 共用 IO，不建议用户自行使用，如需 JTAG 调试，请使用 USB DEBUG 接口。

LED 灯

5 个发光 LED 灯，1 个自定义 LED 灯接在 PS 端，1 个 DONE 配置指示灯，1 个电源指示灯，2 个数据传输指示灯。

按键

2 个按键，1 个用户自定义按键接在 PS 端，1 个全局复位按键。

拨码开关

1 个 2 位拨码开关用于启动模式切换，支持 QSPI、JTAG、SD 卡三种启动模式。

射频 SMA 接口

4 个低插损 SMA 接口（内孔），2 个用于接收无线电信号，2 个用于发射无线电信号。

外部参考时钟输入接口

1 个外部参考时钟输入接口用于给 AD9361 芯片提供时钟参考。

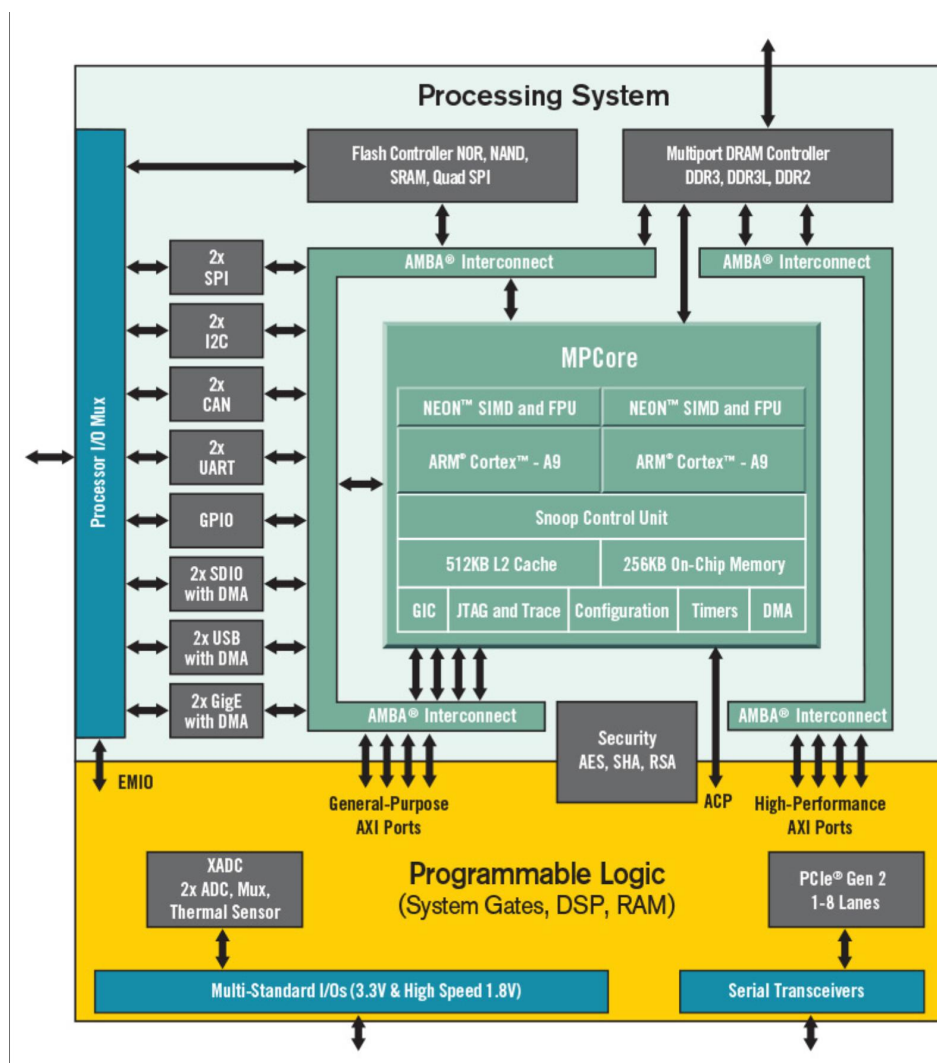
外部本振输入接口

2 个外部本振输入接口，用于接收和发射的外部本振输入。

二、ZYNQ 芯片

开发板使用的是 Xilinx 公司的 Zynq7000 系列的芯片，型号为 XC7Z020-1CLG400I。Zynq7020 芯片的 PS 系统集成了 2 个 ARM Cortex-A9 处理器，速度高达 1Ghz。

Zynq7020 芯片支持 32 位的 DDR3，DDR3L，LPDDR3 存储芯片，在 PS 端带有丰富的接口，如 SPI、UART、ETH、USB、QSPI、SD 等。PL 端内部含有丰富的可编程逻辑单元，DSP 和内部 RAM。Zynq7020 芯片的总体框图如下图所示：



Logic Cells 85K

Look-Up Tables (LUTs) 53,200

CLB Flip-Flops 106,400

DSP Slices 220

Block RAM 4.9Mb

94 IOs, 8 MIOs

47 Pairs LVDS

100 Voltage Adjustable IOs

AD Converter x 2, 1Mbps

Speed Grade -1, Industrial Grade

Working Temperature -40 °C to 85 °C

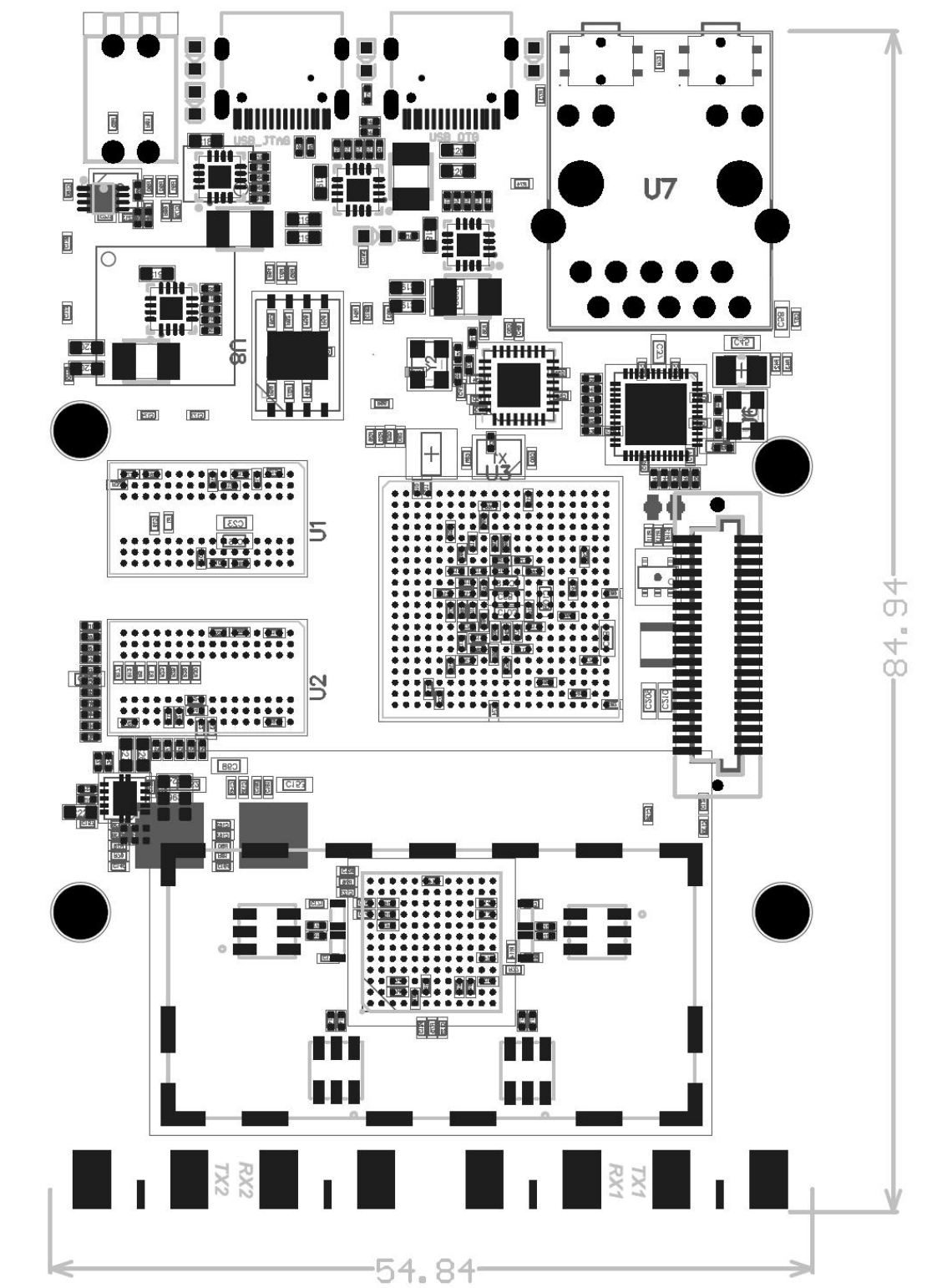
三、DDR 芯片

K210 开发板上配有 2 片 Micron(美光) 的 512MB 的 DDR3 芯片, 型号为 MT41K256M16TW-107, 为 32 位的数据总线宽度和 1GB 的容量。PS 端的 DDR3 SDRAM 的最高运行速度可达 533MHz(数据速率 1066Mbps), 2 片 DDR3 存储系统直接连接到了 PS 的 BANK502 的存储器接口上。

四、QSPI Flash

K210 开发板配有 1 片 256MBit 大小的 SINGLE-SPI FLASH 芯片组成 4 位带宽数据总线, FLASH 型号为 W25Q256FVEIG, 它使用 3.3V CMOS 电压标准。由于 QSPI FLASH 的非易失特性, 在使用中, 它可以作为系统的启动设备来存储系统的启动镜像。这些镜像主要包括 FPGA 的 bit 文件、ARM 的应用程序代码以及其它的用户数据文件。QSPI FLASH 与 FPGA 连接如下图所示。

六、结构图



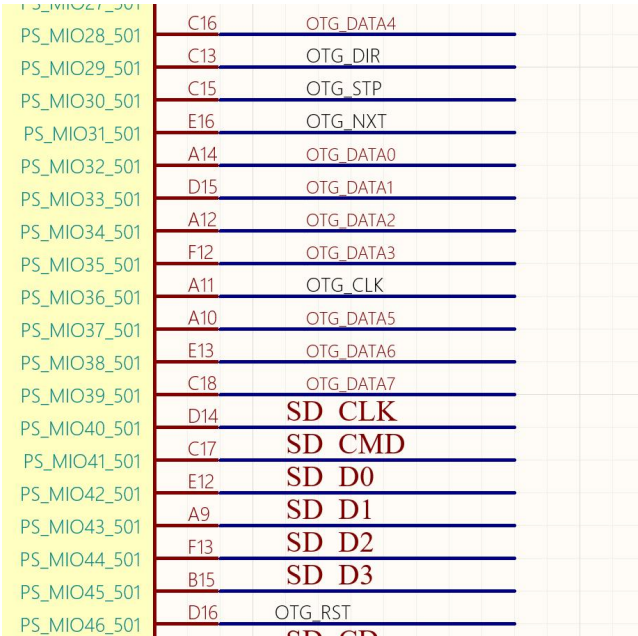
七、BTB 连接器管脚定义

BTB 连接器连接到 Zynq 的 BANK35，电平规格为 3.3V。连接器型号为 61083-044402LF。

管脚编号	FPGA 管脚	管脚编号	FPGA 管脚	管脚编号	FPGA 管脚	管脚编号	FPGA 管脚
1	VCC3V3	11	PL_RXD0	21	L19	31	H20
2	GND	12	USB_ID	22	E18	32	F16
3	VCC3V3	13	H15	23	L20	33	G19
4	GND	14	H18	24	E19	34	D19
5	VCC3V3	15	G15	25	K19	35	G20
6	GND	16	J18	26	E17	36	D20
7	VCC3V3	17	M19	27	J19	37	F19
8	GND	18	H17	28	D18	38	C20
9	PL_TXD0	19	M20	29	J20	39	F20
10	USB_5V	20	H16	30	F17	40	B20

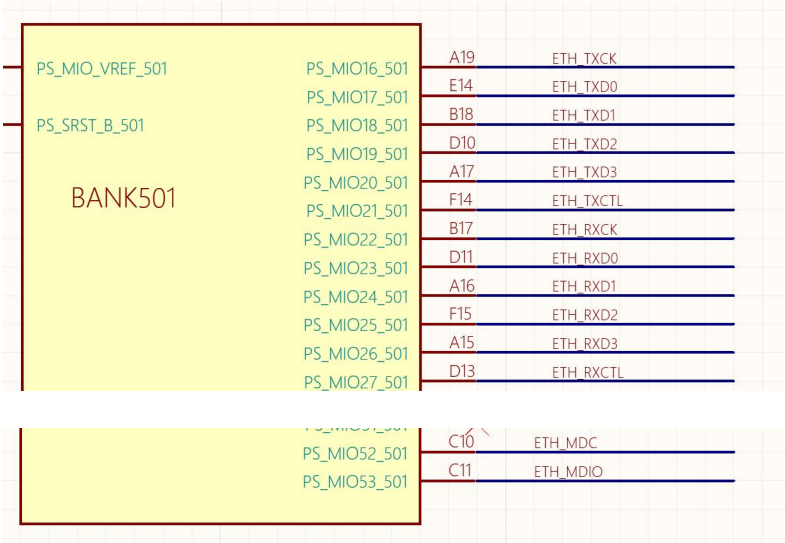
八、USB 2.0 接口

K210 开发板上有一个 USB 2.0 接口，支持 OTG 模式，USB PHY 芯片型号为 USB3320C，与 Zynq 连接如下图所示：



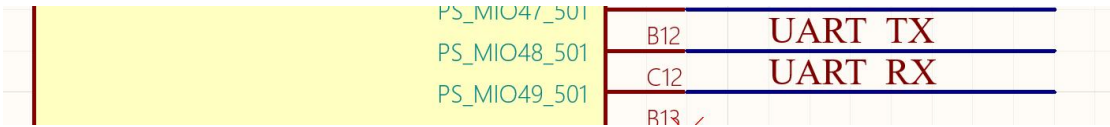
九、千兆以太网接口

K210 开发板上有一个千兆以太网接口，支持 10/100M/1000Mbps 网络传输速率，ETH PHY 芯片型号为 RTL8211E-VL，接口为 RGMII，连接于 PS 端。用于和电脑或其它网络设备进行以太网数据交换，与 Zynq 连接如下图所示：



十、UART 接口

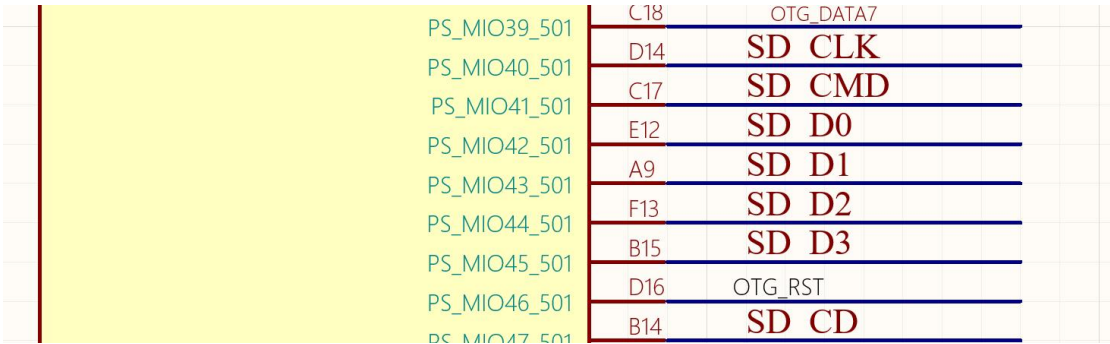
K210 开发板引出一个 UART 接口在 PS 端，编号为 UART1，其中 TXD 接 MIO48,RXD 接 MIO49，与 Zynq 连接如下图所示：



十一、SD 卡槽

K210 开发板包含了一个 Micro 型的 SD 卡接口，以提供用户访问 SD 卡存储器，用于存储 Zynq7020 芯片的 B00T 程序，Linux 操作系统内核，文件系统以及其它的用户数据文件。

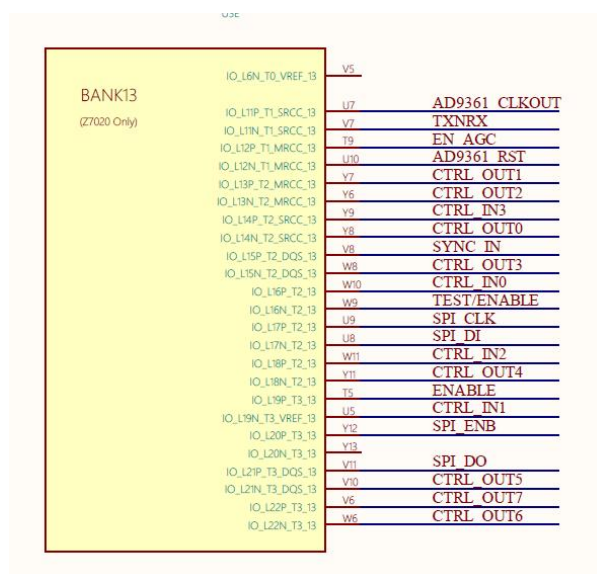
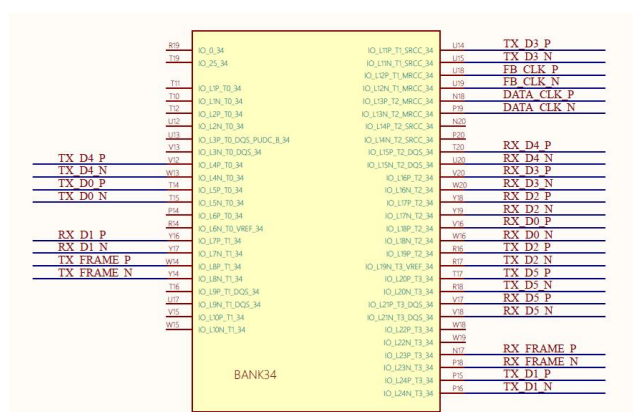
SDIO 信号与 Zynq7020 的 PS BANK501 的 IO 信号相连，因为 501 的 VCCI0 设置为 1.8V，但 SD 卡的数据电平为 3.3V，其中 CD 引脚连接 MIO47，我们这里通过 TXS02612 电平转换器来连接。Zynq7020 PS 和 SD 卡连接器的原理图如下图所示。



十二、AD9361

AD9361，一种高性能、高度集成的射频收发器，旨在用于射频应用，例如 3G 和 4G 基站和测试设备应用，以及软件定义的无线电。其可编程性和宽带能力使其成为广泛的收发器应用的理想选择。该器件将 RF 前端与灵活的混合信号基带部分和集成频率合成器相结合，通过为处理器或 FPGA 提供可配置的数字接口来简化设计。AD9361 芯片的工作频率范围为 70 MHz 至 6 GHz，涵盖大多数许可和非许可频段。该芯片通过改变采样率、数字滤波器和抽取来支持从小于 200 kHz 到 56 MHz 的通道带宽，这些都可以在 AD9361 本身内进行编程。

K210 开发板与 AD9361 在布线上使用 1vds25 规范同时兼容 1vcmos25 规范，使用 spi 接口对芯片进行如收发本振，采样率，带宽等进行配置。使用 1vds 或者 cmos 对数据进行收发，在顶层使用 libiio 标准库与上位机进行通信。其中 1vds 引脚与 Zynq7020 的 bank34 进行连接 vcco 为 2.5v, spi 及控制引脚与 Zynq7020 的 bank13 进行连接 vcco 为 2.5v, 连接图如下：



射频技术指标如下:

1. 射频频率 70MHZ~6GHZ
2. ADC/DAC 采集: 12 位
3. 2 路 TX, 2 路 RX, 工作方式双工与半双工
4. RF 匹配电阻: 50 欧姆
5. 瞬时带宽: 56MHZ
6. 采样率

在 CMOS 模式下:

- 最大数据速率为 122.88 Msps (双端口半双工, DDR 模式),
- 最大 DATA-CLK 速率为 61.44 MHz,

在 LVDS 模式下:

- 最大数据速率为 122.88 Msps (双端口全双工),
- 最大 DATA-CLK 速率为 245.76 MHz,

射频发射技术指标:

本振范围: 70MHZ-6000MHZ

发射功率: <5dbm 1GHZ CW 模式

带宽: 200KHZ-56MHZ

EVM: <2% 5dBm @20MHz

增益范围: >80db

增益步长: 0.25db

边带抑制: 40dbc

本振抑制: 60dbc

射频接收技术指标:

本振范围: 70MHZ-6000MHZ

接收灵敏度: -90dBm@20MHz

带宽: 200KHZ-56MHZ

EVM: <2% 5dBm @20MHz

增益范围: >60db

增益步长: 1db

噪声系数: <8db

IIP3: -25dbm

Vivado 中 xdc 定义如下

1. lvds25

```
# constraints
# ad9361

set_property -dict {PACKAGE-PIN N18 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_clk-in-p] ;
set_property -dict {PACKAGE-PIN N17 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_frame-in-p] ;
set_property -dict {PACKAGE-PIN P18 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_frame-in-n] ;
set_property -dict {PACKAGE-PIN V16 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-p[0]] ;
set_property -dict {PACKAGE-PIN W16 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-n[0]] ;
set_property -dict {PACKAGE-PIN Y16 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-p[1]] ;
set_property -dict {PACKAGE-PIN Y17 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-n[1]] ;
set_property -dict {PACKAGE-PIN Y18 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-p[2]] ;
set_property -dict {PACKAGE-PIN Y19 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-n[2]] ;
set_property -dict {PACKAGE-PIN V20 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-p[3]] ;
set_property -dict {PACKAGE-PIN W20 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-n[3]] ;
set_property -dict {PACKAGE-PIN T20 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
```

```

[get_ports rx_data-in-p[4]] ;
set_property -dict {PACKAGE-PIN U20 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-n[4]] ;
set_property -dict {PACKAGE-PIN V17 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-p[5]] ;
set_property -dict {PACKAGE-PIN V18 IOSTANDARD LVDS-25 DIFF_TERM TRUE}
[get_ports rx_data-in-n[5]] ;
set_property -dict {PACKAGE-PIN U18 IOSTANDARD LVDS-25} [get_ports
tx_clk-out-p] ;
set_property -dict {PACKAGE-PIN U19 IOSTANDARD LVDS-25} [get_ports
tx_clk-out-n] ;
set_property -dict {PACKAGE-PIN W14 IOSTANDARD LVDS-25} [get_ports
tx_frame-out-p] ;
set_property -dict {PACKAGE-PIN Y14 IOSTANDARD LVDS-25} [get_ports
tx_frame-out-n] ;
set_property -dict {PACKAGE-PIN T14 IOSTANDARD LVDS-25} [get_ports
tx_data-out-p[0]] ;
set_property -dict {PACKAGE-PIN T15 IOSTANDARD LVDS-25} [get_ports
tx_data-out-n[0]] ;
set_property -dict {PACKAGE-PIN P15 IOSTANDARD LVDS-25} [get_ports
tx_data-out-p[1]] ;
set_property -dict {PACKAGE-PIN P16 IOSTANDARD LVDS-25} [get_ports
tx_data-out-n[1]] ;
set_property -dict {PACKAGE-PIN R16 IOSTANDARD LVDS-25} [get_ports
tx_data-out-p[2]] ;
set_property -dict {PACKAGE-PIN R17 IOSTANDARD LVDS-25} [get_ports
tx_data-out-n[2]] ;
set_property -dict {PACKAGE-PIN U14 IOSTANDARD LVDS-25} [get_ports
tx_data-out-p[3]] ;
set_property -dict {PACKAGE-PIN U15 IOSTANDARD LVDS-25} [get_ports
tx_data-out-n[3]] ;
set_property -dict {PACKAGE-PIN V12 IOSTANDARD LVDS-25} [get_ports
tx_data-out-p[4]] ;
set_property -dict {PACKAGE-PIN W13 IOSTANDARD LVDS-25} [get_ports
tx_data-out-n[4]] ;
set_property -dict {PACKAGE-PIN T17 IOSTANDARD LVDS-25} [get_ports
tx_data-out-p[5]] ;
set_property -dict {PACKAGE-PIN R18 IOSTANDARD LVDS-25} [get_ports
tx_data-out-n[5]] ;

set_property -dict {PACKAGE-PIN Y8 IOSTANDARD LVCMOS25} [get_ports
gpio-status[0]] ;
set_property -dict {PACKAGE-PIN Y7 IOSTANDARD LVCMOS25} [get_ports
gpio-status[1]] ;
set_property -dict {PACKAGE-PIN Y6 IOSTANDARD LVCMOS25} [get_ports
gpio-status[2]] ;
set_property -dict {PACKAGE-PIN W8 IOSTANDARD LVCMOS25} [get_ports
gpio-status[3]] ;
set_property -dict {PACKAGE-PIN Y11 IOSTANDARD LVCMOS25} [get_ports
gpio-status[4]] ;

```

```

set_property -dict {PACKAGE_PIN V10 IOSTANDARD LVCMOS25} [get_ports
gpio-status[5]]
;
set_property -dict {PACKAGE_PIN W6 IOSTANDARD LVCMOS25} [get_ports
gpio-status[6]]
;
set_property -dict {PACKAGE_PIN V6 IOSTANDARD LVCMOS25} [get_ports
gpio-status[7]]
;
set_property -dict {PACKAGE_PIN W10 IOSTANDARD LVCMOS25} [get_ports
gpio-ctl[0]]
;
set_property -dict {PACKAGE_PIN U5 IOSTANDARD LVCMOS25} [get_ports
gpio-ctl[1]]
;
set_property -dict {PACKAGE_PIN W11 IOSTANDARD LVCMOS25} [get_ports
gpio-ctl[2]]
;
set_property -dict {PACKAGE_PIN Y9 IOSTANDARD LVCMOS25} [get_ports
gpio-ctl[3]]
;
set_property -dict {PACKAGE_PIN T5 IOSTANDARD LVCMOS25} [get_ports
enable]
;
set_property -dict {PACKAGE_PIN V7 IOSTANDARD LVCMOS25} [get_ports
txrx]
;
set_property -dict {PACKAGE_PIN T9 IOSTANDARD LVCMOS25} [get_ports
gpio-en-agc]
;
set_property -dict {PACKAGE_PIN V8 IOSTANDARD LVCMOS25} [get_ports
gpio-sync]
;
set_property -dict {PACKAGE_PIN U10 IOSTANDARD LVCMOS25} [get_ports
gpio-resetb]
;

set_property -dict {PACKAGE_PIN Y12 IOSTANDARD LVCMOS25 PULLTYPE PULLUP}
[get_ports spi-csn]
;
set_property -dict {PACKAGE_PIN U9 IOSTANDARD LVCMOS25} [get_ports
spi-clk]
;
set_property -dict {PACKAGE_PIN U8 IOSTANDARD LVCMOS25} [get_ports
spi-mosi]
;
set_property -dict {PACKAGE_PIN V11 IOSTANDARD LVCMOS25} [get_ports
spi-miso]
;
# clocks

create_clock -name rx_clk -period 4 [get_ports rx_clk-in-p]

```

2. cmos25

```

# constraints

# ad9361 (SWAP == 0x1)

set_property -dict {PACKAGE_PIN N18 IOSTANDARD LVCMOS25 } [get_ports
rx_clk-in]

set_property -dict {PACKAGE_PIN N17 IOSTANDARD LVCMOS25 } [get_ports

```

```

rx_frame_in]

set_property -dict {PACKAGE_PIN W16 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[0]]

set_property -dict {PACKAGE_PIN V16 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[1]]

set_property -dict {PACKAGE_PIN Y17 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[2]]

set_property -dict {PACKAGE_PIN Y16 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[3]]

set_property -dict {PACKAGE_PIN Y19 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[4]]

set_property -dict {PACKAGE_PIN Y18 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[5]]

set_property -dict {PACKAGE_PIN W20 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[6]]

set_property -dict {PACKAGE_PIN V20 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[7]]

set_property -dict {PACKAGE_PIN U20 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[8]]

set_property -dict {PACKAGE_PIN T20 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[9]]

set_property -dict {PACKAGE_PIN V18 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[10]]

set_property -dict {PACKAGE_PIN V17 IOSTANDARD LVCMOS25 } [get_ports
rx_data_in[11]]

```

set_property	-dict	{PACKAGE_PIN	U18	IOSTANDARD	LVC MOS25}	[get_ports tx_clk_out]
set_property	-dict	{PACKAGE_PIN	W14	IOSTANDARD	LVC MOS25}	[get_ports tx_frame_out]
set_property	-dict	{PACKAGE_PIN	T15	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[0]]
set_property	-dict	{PACKAGE_PIN	T14	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[1]]
set_property	-dict	{PACKAGE_PIN	P16	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[2]]
set_property	-dict	{PACKAGE_PIN	P15	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[3]]
set_property	-dict	{PACKAGE_PIN	R17	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[4]]
set_property	-dict	{PACKAGE_PIN	R16	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[5]]
set_property	-dict	{PACKAGE_PIN	U15	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[6]]
set_property	-dict	{PACKAGE_PIN	U14	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[7]]
set_property	-dict	{PACKAGE_PIN	W13	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[8]]
set_property	-dict	{PACKAGE_PIN	V12	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[9]]
set_property	-dict	{PACKAGE_PIN	R18	IOSTANDARD	LVC MOS25}	[get_ports tx_data_out[10]]
set_property	-dict	{PACKAGE_PIN	T17	IOSTANDARD	LVC MOS25}	[get_ports

```
tx_data_out[11]]

set_property -dict {PACKAGE_PIN Y8 IOSTANDARD LVCMOS25} [get_ports
gpio_status[0]]

set_property -dict {PACKAGE_PIN Y7 IOSTANDARD LVCMOS25} [get_ports
gpio_status[1]]

set_property -dict {PACKAGE_PIN Y6 IOSTANDARD LVCMOS25} [get_ports
gpio_status[2]]

set_property -dict {PACKAGE_PIN W8 IOSTANDARD LVCMOS25} [get_ports
gpio_status[3]]

set_property -dict {PACKAGE_PIN Y11 IOSTANDARD LVCMOS25} [get_ports
gpio_status[4]]

set_property -dict {PACKAGE_PIN V10 IOSTANDARD LVCMOS25} [get_ports
gpio_status[5]]

set_property -dict {PACKAGE_PIN W6 IOSTANDARD LVCMOS25} [get_ports
gpio_status[6]]

set_property -dict {PACKAGE_PIN V6 IOSTANDARD LVCMOS25} [get_ports
gpio_status[7]]

set_property -dict {PACKAGE_PIN W10 IOSTANDARD LVCMOS25} [get_ports
gpio_ctl[0]]

set_property -dict {PACKAGE_PIN U5 IOSTANDARD LVCMOS25} [get_ports
gpio_ctl[1]]

set_property -dict {PACKAGE_PIN W11 IOSTANDARD LVCMOS25} [get_ports
gpio_ctl[2]]

set_property -dict {PACKAGE_PIN Y9 IOSTANDARD LVCMOS25} [get_ports
gpio_ctl[3]]
```

```

set_property -dict {PACKAGE_PIN T9 IOSTANDARD LVCMOS25} [get_ports
gpio-en-agc]

set_property -dict {PACKAGE_PIN U10 IOSTANDARD LVCMOS25} [get_ports
gpio-resetb]

set_property -dict {PACKAGE_PIN T5 IOSTANDARD LVCMOS25} [get_ports enable]

set_property -dict {PACKAGE_PIN V7 IOSTANDARD LVCMOS25} [get_ports txnrx]

#NO USE

set_property -dict {PACKAGE_PIN R19 IOSTANDARD LVCMOS25 PULLTYPE PULLUP}
[get_ports iic-scl]

set_property -dict {PACKAGE_PIN T19 IOSTANDARD LVCMOS25 PULLTYPE PULLUP}
[get_ports iic-sda]

set_property -dict {PACKAGE_PIN Y12 IOSTANDARD LVCMOS25 PULLTYPE PULLUP}
[get_ports spi-csn]

set_property -dict {PACKAGE_PIN U9 IOSTANDARD LVCMOS25} [get_ports spi-clk]

set_property -dict {PACKAGE_PIN U8 IOSTANDARD LVCMOS25} [get_ports
spi-mosi]

set_property -dict {PACKAGE_PIN V11 IOSTANDARD LVCMOS25} [get_ports
spi-miso]

#NO USE

set_property -dict {PACKAGE_PIN T10 IOSTANDARD LVCMOS25} [get_ports
p1-spi-clk-o]

set_property -dict {PACKAGE_PIN T12 IOSTANDARD LVCMOS25} [get_ports
p1-spi-miso]

set_property -dict {PACKAGE_PIN T11 IOSTANDARD LVCMOS25} [get_ports
p1-spi-mosi]

set_property -dict {PACKAGE_PIN U7 IOSTANDARD LVCMOS25} [get_ports

```

```
clk-out]

create_clock -name rx_clk -period 16.27 [get_ports rx_clk_in]

# probably gone in 2016.4

create_clock      -name      clk_fpga_0      -period      10      [get_pins
"i-system-wrapper/system-i/sys_ps7/inst/PS7-i/FCLKCLK[0]" ]

create_clock      -name      clk_fpga_1      -period      5      [get_pins
"i-system-wrapper/system-i/sys_ps7/inst/PS7-i/FCLKCLK[1]" ]

create_clock -name spi0_clk      -period 40      [get_pins -hier */EMIOSPI0SCLK0]

set_input_jitter clk_fpga_0 0.3

set_input_jitter clk_fpga_1 0.15

set_property SEVERITY {Warning} [get_drc_checks NSTD-1]

set_property SEVERITY {Warning} [get_drc_checks RTSTAT-1]

set_property SEVERITY {Warning} [get_drc_checks UCIO-1]
```

后续如有任何问题都可通过 neptunesdr@163.com 与我们联系。

感谢您支持杭州涅普科技有限责任公司！祝您后续应用顺利！

Neptune SDR

©Copyright Neptune SDR 2023

杭州涅普科技有限责任公司

地址：中国 浙江 杭州

邮箱：neptunesdr@163.com

淘宝：[涅普科技 SDR](#)